

人間計算可能なパスワードに対する 長短期記憶ニューラルネットワークによる 安全性評価

1TE19092S 丸野美矩人
令和5年2月

九州大学 工学部 電気情報工学科

概要

近年、インターネットサービスの利用の増加に伴い、複数のオンラインアカウントならびに、それぞれのアカウントごとに異なるパスワードを保有するようになってきている。これらのパスワードの安全性を確保するためには、アカウントごとに関連性を持たない異なるパスワードを使用することが必要になる。しかし、パスワードの現状は同じパスワードを複数のアカウントで併用していたり、名前や誕生日などの個人情報やキーボードの配列順などの単純な文字列がパスワードとして用いられている。これらのパスワードは個人情報から推定されたり、流出したりした際に他のパスワードも流出してしまう危険性がある。そこで提案されたものが、人間計算可能なパスワードである。

人間計算可能なパスワードとは、Manuel Blum らによって提案されたチャレンジレスポンス方式のパスワード認証で、ユーザーが認証の際に簡単な計算を暗算で行うため、人間計算可能なパスワードと呼ばれる。このパスワードは事前知識として画像と数字の対応を学習・記憶する必要があり、その記憶をもとに示された関数を解くことでユーザー認証を可能にする。

既存研究は、多層パーセプトロン (MLP: MUltilayer Perceptron) を用いて、画像から数値への対応と暗算に用いる合成関数の出力が予測可能であるかについて実験を行い、予測精度は 10% 前後であると評価している。また、関数を線形にした場合、予測精度が約 80% に向上したと示している。しかし、1 つのモデルでしか検証されていないため、他の深層学習モデルでの評価が課題である。

本研究では、多層パーセプトロンと同様に数値を扱うことのできる深層学習モデルとして、長短期記憶ニューラルネットワーク (LSTM: Long Short Term Memory) を用いて人間計算可能なパスワードの安全性評価ならびに MLP での実験結果との比較を行った。その結果、合成関数の予測精度は 10% 前後となり、人間計算可能なパスワードにおいて用いられている合成関数は、安全であることが分かった。一方で関数を線形にした場合の予測精度は 90% を超え、MLP による予測精度より 10~20% ほど高くなった。これより、出力に関係する値が演算のたびに変化することが、予測精度の低下につながっていることが分かった。また、既存研究と本研究の結果から、深層学習解析が人間計算可能なパスワードの安全性評価として有効であることを再検証できた。

Security Assessment of Human Computable Passwords by Using LSTM

Abstract

In recent years, with the increase in the use of Internet services, people have multiple online accounts and different passwords for each account. To ensure the security of these passwords, it is necessary to use different passwords that are not related to each other for each account. However, the current state of passwords is that the same password is used in conjunction with multiple accounts, or a simple string of personal information such as name, birthday, or keyboard sequence order is used as a password. These passwords are at risk of being extrapolated from personal information, or when they are leaked, other passwords may be leaked as well. What was proposed there is a human computable password.

Human computable passwords are challenge-response password authentication proposed by Manuel Blum et al. They are called human computable passwords because the user performs a simple calculation by rote during authentication. This password requires learning and memorizing the correspondence between images and numbers as prior knowledge, and then solving the indicated function based on that memory enables user authentication.

Existing research has experimented with the multilayer perceptron (MLP: Multilayer Perceptron) to determine whether the output of the composite function used for image-to-number correspondence and implicit computation can be predicted, and has evaluated the prediction accuracy to be around 10%. It also indicates that when the function is linear, the prediction accuracy has improved to about 80%. However, since it has only been validated with one model, evaluation with other deep learning models is an issue.

In this study, we evaluated the security of human computable passwords using the Long Short Term Memory Neural Network (LSTM: Long Short Term Memory), as a deep learning model that can handle numerical values in the same way as the multilayer perceptron, and compared the results with experimental results using MLP. The results showed that the prediction accuracy of the composite function was around 10%, indicating that the composite function used in human computable passwords is secure. On the other hand, when the function is linear, the prediction accuracy exceeds 90%, which is 10~20% higher than the prediction accuracy by MLP. From this, it was found that the values related to the output change with each calculation, which leads to a decrease in prediction accuracy. The results of the existing studies and this study also reaffirm the effectiveness of deep learning

analysis as a security assessment for human-computable passwords.

目次

概要	i
表目次	v
図目次	v
1 緒論	1
1.1 研究背景	1
1.2 本論文の貢献	1
1.3 本論文の構成	2
2 背景	2
2.1 パスワードの現状	2
2.2 パスワードの定義	2
2.3 LSTM	3
3 既存研究	5
3.1 人間計算可能なパスワード	5
3.2 MLP による人間計算可能なパスワードの安全性評価	7
4 LSTM による人間計算可能なパスワードの安全性評価	9
4.1 合成関数 $f \circ \sigma$ の予測精度	9
4.2 関数 f ならびに関数 g の予測精度	9
4.3 関数 f の項の数を変化させた場合の予測精度	12
5 結論	14
謝辞	16
参考文献	17
付録	18

表目次

1	MLP による合成関数 $f \circ \sigma$ の予測精度	7
2	MLP による関数 f の予測精度	8
3	MLP による関数 g の予測精度	8
4	合成関数 $f \circ \sigma$ の予測精度	9
5	関数 f の予測精度	11
6	関数 g の予測精度	12
7	関数 h_1 の予測精度	13
8	関数 h_2 の予測精度	14
9	人間計算可能なパスワードと CAPTCHA の比較	23

図目次

1	ニューラルネットワークの構成	4
2	LSTM の構成	4
3	画像とマッピング関数の対応表	6
4	人間計算可能なパスワード	7
5	$n=26$ 、データ数 10000 における予測精度	9
6	$n=26$ 、データ数 50000 における予測精度	10
7	$n=26$ 、データ数 100000 における予測精度	10
8	関数 f の予測精度	11
9	関数 g の予測精度	12
10	関数 h_1 の予測精度	13
11	関数 h_2 の予測精度	14

1 緒論

1.1 研究背景

現在利用されているパスワードは、ユーザーの個人的な情報と関連性のあるものやキーボードの配列順などの単純な文字列が多く用いられている。また、多くの人が複数のアカウントで同じパスワードを使いまわしているという現状がある。これらのことは、個人情報から推定される危険性があり、1つのパスワードが漏洩すると、他のアカウントのパスワードも漏洩することになり、パスワードの安全性という観点において懸念が残る。そこで提案されたものが、人間計算可能なパスワードである。

人間計算可能なパスワードは2016年にManuel Blumらによって提案されたチャレンジレスポンス方式のユーザー認証である。人間計算可能なパスワードは事前知識として画像と数字の対応を学習・記憶する必要がある。ユーザーは記憶した画像と数字の対応をもとに、パスワード入力画面に提示された14枚の画像・関数を計算する。計算結果をパスワードとして入力することで、認証が可能となる。この提示される14枚の画像はパスワード認証を行うたびに、ランダムに選択されるためパスワードとしての安全性は確保されることになる。

既存研究は、この人間計算可能なパスワード、すなわち提示される関数の計算結果をどれくらいの精度で予測できるかを多層パーセプトロンを用いて実験を行ったものである。結果として、予測精度は10%前後であり、予測はできないという結論になっている。また、関数を非線形から線形に変更した場合は予測精度が約80%となり、入力の値を直接演算に使う場合と一定のランダム性を含んでいる場合で、予測精度に大きな差が出ることが分かった。

そこで、本研究では多層パーセプトロンと同じく数値を扱うことのできる深層学習モデルとして、LSTMを用いて人間計算可能なパスワードの安全性評価を行った。結果として、既存研究と同様に予測精度は10%前後となり、安全であるという結論に至った。また、関数を非線形から線形にした場合、予測精度は約95%となり、既存研究と比べ高い値となった。さらに関数を簡略化したものや項の数を変更したものでも実験を行い、いずれも予測精度は約15%で、計算に使われる値が計算のたびに変わることが予測精度の低下につながっているということも分かった。

1.2 本論文の貢献

本論文ではまず、人間計算可能なパスワードの背景知識として、パスワードの現状ならびにパスワードの定義について説明する。また、人間計算可能な条件について説明し、人間計算可能なパスワードを紹介する。

さらに、既存研究として多層パーセプトロンを用いた人間計算可能なパスワードの安全性

評価を紹介する。この手法を LSTM で実装し、人間計算可能なパスワードの安全性評価を行い、MLP での結果と比較する。また、人間計算可能なパスワードで使用した関数を簡略化したもの、関数の項の数を変更したものについても同様の安全性評価を行い、LSTM の予測精度の低下の要因についても確認する。

1.3 本論文の構成

第2章では、パスワードの現状やパスワードの定義、評価に用いた LSTM など研究背景について記す。第3章では、既存研究について記す。第4章では、LSTM による人間計算可能なパスワードの安全性評価の結果を記す。第5章では、本論文のまとめを記す。また、付録では、人間計算可能なパスワードと CAPTCHA の比較を記す。

2 背景

2.1 パスワードの現状

近年、インターネットサービスの利用の増加に伴い、多くのオンラインアカウントならびに、それぞれのアカウントごとに異なるパスワードを保有するようになってきている。これらのパスワードの安全性を確保するためには、アカウントごとに関連性をもたないパスワードや法則性がない文字列にする必要がある。しかし、2022年11月にセキュリティ企業が実施した実態調査によると、8割を超える人が Web サービスやアプリにおいて、「少なくとも1つ以上は同じパスワードを使いまわしている」と回答している。さらに全体の半数近くは5つ以上のサイトでパスワードを使いまわしているという結果であった [1]。

2.2 パスワードの定義

人間計算可能なパスワードはチャレンジに対する回答によって認証される。このパスワードはパスワードスキーマによって生成され、そのパスワードスキーマは、チャレンジと鍵のペアをパスワードに対応付けるための人間が計算可能なアルゴリズムである。

パスワードは必要な時に簡単に作ることができ、敵対者がパスワードユーザーのスキーマを理解し、その他の少数のアカウントのパスワードを知ったとしても、偽装することが難しいものであるべきである。

より一般的に次のようなスキーマを求める。分析可能であること、公開可能であること、人間が使用可能であること、安全であること、自己リハーサルが可能であること [2]。

1. 分析可能：スキーマがチューリングマシンによって実行できるほど正確に定義されていること

2. 公開可能：ユーザーの秘密鍵を除くパスワードスキーマを公開することができること
3. 人間が使用可能：次の3つの条件を満たすこと（1）秘密鍵を除くスキーマ自体をユーザーが数分程度で理解できること。（2）秘密鍵の記憶ならびに生成が、最大で1時間、可能であれば30分以内でできること。（3）パスワードの生成・再生成が最大で1分、可能であれば20秒以内でできること。
4. 安全：セキュリティ Q を持つこと。セキュリティ Q を持つとは、Q よりも少ないチャレンジに対する応答を見た敵対者が、ランダムに選択されたチャレンジに対して正しい応答を推測する確率が $1/10$ 未満であること。
5. 自己リハーサル可能：ランダムなチャレンジに対応する過程で、ユーザーがスキーマと鍵のあらゆる面をリハーサルすることが可能であること。

2.3 LSTM

2.3.1 深層学習

深層学習とは、多層構造のニューラルネットワークを用いて、入力されたデータのパターンを学習し、分類や識別していくためにパターン学習を行うものである [5]。深層学習を行う際、学習する回数や入力するデータの数を多くすることで、識別精度を上げることが可能である。

図1はニューラルネットワークの構成を表したもので、丸のことをノード、ノードの列を層と呼ぶ。ニューラルネットワークは入力層・中間層・出力層から形成されており、中間層を多層化することで、情報伝達と処理の増加、予測精度を向上させたものが深層学習である [6]。

2.3.2 LSTM

LSTM は主に忘却ゲート、入力ゲート、出力ゲートの3つのゲートから成り立っている [8]。それぞれのゲートにおける構造を図2に示す [7]。図2における一番左の σ （シグモイド関数）のラインが忘却ゲートで、前セルからの長期記憶に対して情報の取捨選択を行うものである。この忘却ゲートで不要な情報を捨てることで、長い時系列データも扱えるようになった要因の一つである。入力ゲートでは短期記憶 $h_t - 1$ と入力 x_t からなる入力データを、どの信号をどのくらいの重みで長期記憶に保存するか制御を行う。入ってきた情報をそのまま流すのではなく、 $\tanh$ によって端的な形として C'_t が出力される。出力ゲートでは、忘却ゲートおよび入力ゲートで取捨選択された情報を、そのまま長期記憶として C_t を出力する。短期記憶部分も長期記憶と合わせることで、端的で使いやすいものに変換することができる。また、 h_t を更新する際にシグモイド関数により O_t が出力され、短期記憶出力 h_t に必要な信号だけを適切に伝搬するように制御している [9]。

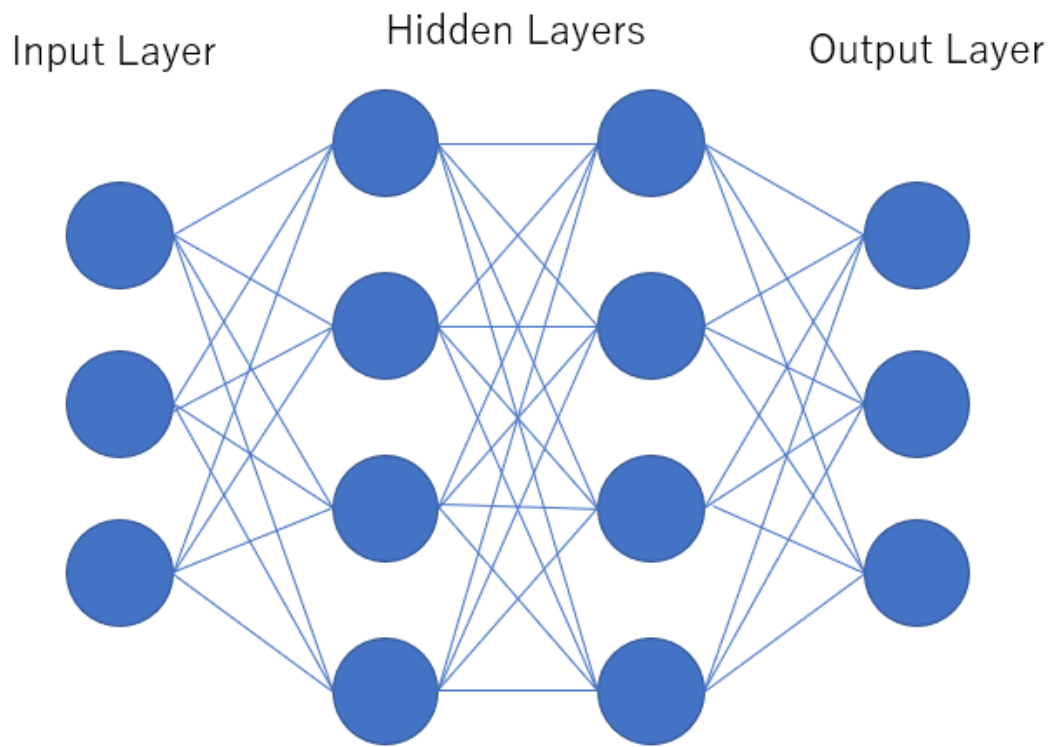


図1 ニューラルネットワークの構成

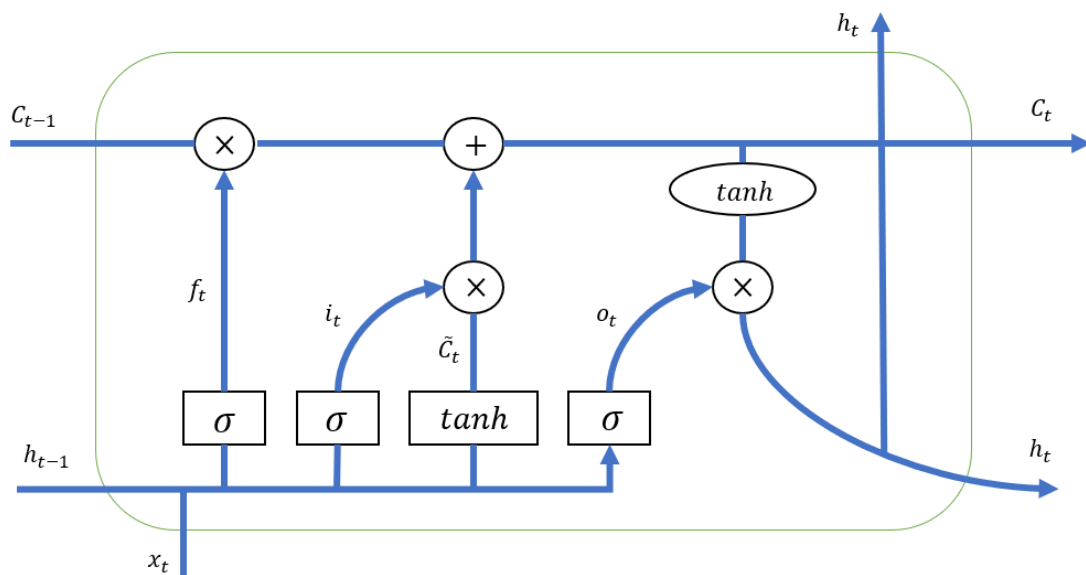


図2 LSTM の構成

3 既存研究

3.1 人間計算可能なパスワード

3.1.1 人間計算の定義

人間の計算は前処理段階の PRPE（スキーマ自体の学習、鍵の生成と記憶）と処理段階の PROC（ある入力に対して関連する鍵を持つスキーマの実行）で構成される。ここでは、人間計算可能なパスワードにおける人間を以下のようにモデル化する。

1. 人間とは一種のチューリングマシンであり、通常のメモリの代わりに長期用と短期用の 2 つのランダムアクセスメモリが搭載されたものである。
2. 長期記憶は無限大の可能性を持っており、その上限は人間の寿命と記憶に情報を保存するのにかかる時間だけである。長期記憶への情報の保存は時間がかかるが、長期記憶からの情報の読み出しは比較的早く行うことができる。
3. 長期記憶は、前処理段階で書き込みと読み出しの両方が行われ、処理段階では読み出しのみが行われる。
4. 短期記憶は高速であるが小さく、通常 2～3 個のチャンクを格納し、各チャンクは数字や文字、画像などのアイテムへのポインタである。このチャンクは明確に定義されたオブジェクトであり、長期記憶へのポインタである。
5. スワードでは、入力（チャレンジ）は短期記憶の中の左端の開始位置へポインタを持つ単結合リストとして提示される。チャレンジのどの位置であっても、ポインタは右に移動させたり、読んだり、開始位置にリセットしたりを 1 ステップで行うことができる。ユーザーは長期記憶の中にある単語やフレーズで、左から右は簡単にアクセスできるが、その逆はできない。

3.1.2 人間計算可能な条件

ここで、PRPE（スキーマ自体の学習、鍵の生成と記憶）を MEM（秘密鍵の生成と記憶、ならびに記憶を保持するために要するリハーサル）と COMM（スキーマ自体の学習）に分ける。スキーマと鍵の組み合わせにおいて、以下の条件を満たす場合に人間計算可能であるとみなす。

1. COMM-TIME：スキーマを学習する時間の上限。この時間には、いくつかのチャレンジをパスワードに変換する時間とすべてのリハーサル時間が含まれている。COMM-TIME は最大で 10 分程度でなければならない。
2. PRPE-TIME：スキーマに関連する秘密鍵の生成と記憶、およびに記憶の維持に必要なすべてのリハーサル時間の上限。パスワードの場合、PRPE-TIME は最大で 2 時間

であることが要求される。

3. PROC-TIME：1つの入力に対してスキーマを実行する時間の上限。パスワードの場合、PROC-TIME は最大で1分であることが要求される。
4. スキーマは最大で3つ、可能であれば2つのポインタを長期記憶と短期記憶で使用する。
5. パスワードの場合、長期記憶に保存されるスキーマとそれに関連する鍵は完全に自己リハーサルされる。

3.1.3 人間計算可能なパスワード

チャレンジレスポンス方式のパスワード認証であり、人間（ユーザー）が事前に記憶した秘密をもとに、チャレンジに応答することで認証を行う。今回扱う人間計算可能なパスワードは、1桁チャレンジであり、入力するパスワードに相当するものが0～9のいずれかであることから、1桁チャレンジと呼ばれる。ここでは、画像の枚数 $n = 100$ の条件下で行うものとする。

1. 事前知識

人間計算可能なパスワードを使用するにあたり、ユーザーは事前知識として画像 n 枚（今回は100枚）とそれぞれの画像に対応するマッピング関数 $\sigma(I_i)$ を記憶する必要がある。ユーザーが記憶する画像とマッピング関数は図3で表される。

画像番号	1	2	3		8			16
画像(I)				...		...	...	
$\sigma(I)$	1	2	8		6			5
17		20		26		29		35
	...		...		...		...	
9		8		7		1		3
	45			60		64		n
...		...	...		...		...	
	4			7		0		2

図3 画像とマッピング関数の対応表

2. 1桁チャレンジ [3]

ユーザーはパスワード入力画面に示された14枚の画像をもとに合成関数 $f \circ \sigma$ を計算する。図4は n 枚の画像からランダムに選択された14枚の画像、ならびにユーザーが計算する関数を示している。図4の場合、計算結果である「0」を入力することで認証が可能になる。パスワード入力画面に提示される14枚の画像は、認証のたびに切り替わるため安全性も確保されることになる。



図4 人間計算可能なパスワード

3.2 MLP による人間計算可能なパスワードの安全性評価

3.2.1 合成関数 $f \circ \sigma$ の検証

先行研究では $n = 26, 30, 50, 100$ での実験が行われた [4]。表 1。

表 1 MLP による合成関数 $f \circ \sigma$ の予測精度

データ数	中間層の数	中間層のノード数	予測精度 $n = 26$	予測精度 $n = 30$	予測精度 $n = 50$	予測精度 $n = 100$
1000	3	128,64,32	0.0800	0.0975	0.1033	0.0800
1000	4	128,64,32,16	0.0900	0.0850	0.1133	0.0950
1000	4	256,128,64,32	0.1067	0.0850	0.1033	0.0800
10000	3	128,64,32	0.1017	0.1050	0.0980	0.1060
10000	4	128,64,32,16	0.0987	0.0860	0.0890	0.0905
10000	5	256,128,64,32,16	0.0980	0.0915	0.1017	0.1010
100000	4	128,64,32,16	0.1174	0.1013	0.1012	0.0991
100000	5	256,128,64,32,16	0.1045	0.1041	0.0996	0.0976
100000	6	512,256,128,6,32,16	0.1022	0.1032	0.0990	0.1007

これより、実行したモデル全てで 10% 前後という結果であり、 n の値にかかわらず、関数 $f \circ \sigma$ の答えは予測できないという結論であった。

3.2.2 関数 f のみの場合の検証

表 2 は、マッピング関数 σ を取り除き、関数 f のみの場合の実験結果である。

表 2 MLP による関数 f の予測精度

データ数	中間層の数	中間層のノード数	予測精度
1000	3	128,64,32	0.0900
1000	4	128,64,32,16	0.1100
1000	4	256,128,64,32	0.0850
10000	3	128,64,32	0.0965
10000	4	128,64,32,16	0.0905
10000	5	256,128,64,32,16	0.1070
100000	4	128,64,32,16	0.1026
100000	5	256,128,64,32,16	0.1023
100000	6	512,256,128,64,32,16	0.1297

この結果、予測精度は最も高い場合で 13% となり、関数 f のみの場合においても予測が難しいことが観察された。

3.2.3 関数 g の検証

関数 g は、最初から $x_0 \sim x_9$ のどの値を加えるかが決まっている場合の関数である。表 6 は、一例として関数 g が $g=x_2+x_{12}+x_{13}$ 、 $g=x_5+x_{12}+x_{13}$ の場合の実験結果である。

表 3 MLP による関数 g の予測精度

関数 g	データ数	中間層の数	中間層のノード数	予測精度
$x_2+x_{12}+x_{13}$	10000	3	128,64,32	0.7896
$x_2+x_{12}+x_{13}$	10000	4	128,64,32,16	0.7420
$x_5+x_{12}+x_{13}$	10000	3	128,64,32	0.8630
$x_5+x_{12}+x_{13}$	10000	4	128,64,32,16	0.6155

これより、関数 g は約 80% の精度で予測可能であり、決まった 3 つの単純和は MLP を用いて、容易に予測できることが分かった。

4 LSTM による人間計算可能なパスワードの安全性評価

4.1 合成関数 $f \circ \sigma$ の予測精度

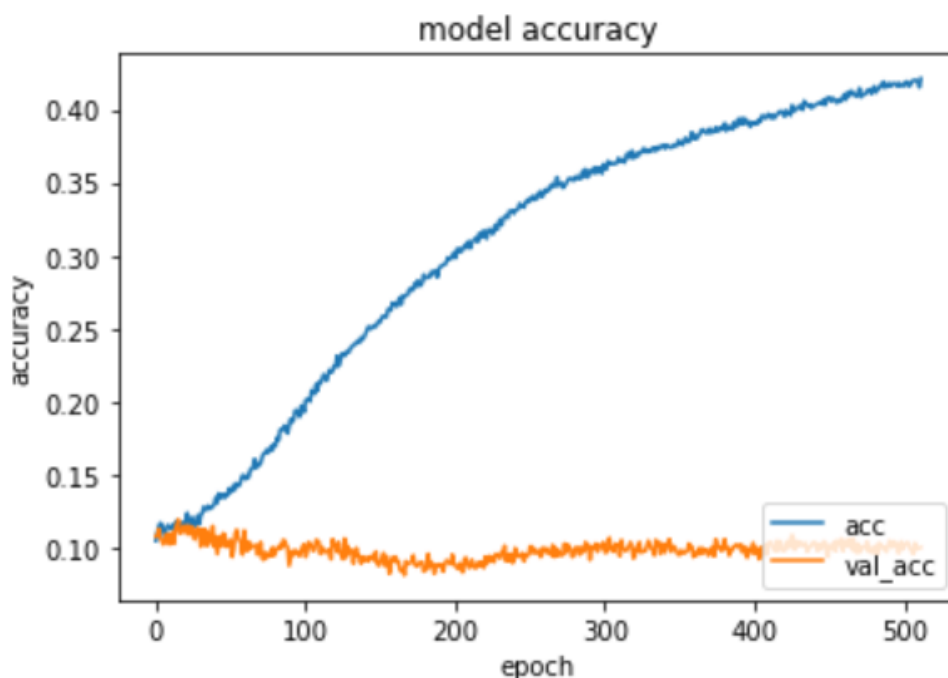
記憶する画像の枚数 $n=26, 50, 100$ のときの実行を行った。以下結果を示す。

今回実行したモデルでは、すべて予測精度は 10% 前後となった。これより、合成関数 $f \circ \sigma$ は、 n の値によらず予測することは難しいと考えられる。以下に $n=26$ における、データ数

表 4 合成関数 $f \circ \sigma$ の予測精度

データ数	予測精度 $n = 26$	予測精度 $n = 50$	予測精度 $n = 100$
10000	0.1002	0.0979	0.0973
50000	0.1121	0.1004	0.0989
100000	0.1152	0.1023	0.1008

10000、50000、100000 のときのそれぞれの学習グラフを示す。

図 5 $n=26$ 、データ数 10000 における予測精度

4.2 関数 f ならびに関数 g の予測精度

4.2.1 関数 f のみの場合の予測精度

マッピング関数 σ を取り除き、関数 f だけの場合についても同様の実行を行った。つまり、 $x_0 \sim x_{13}$ に 0 ～ 9 のランダムな数を代入しただけの場合の予測精度を確かめた。以下にその結果を示す。

この結果、予測精度は約 13% ～ 15% となり、合成関数 $f \circ \sigma$ と比較すると、少し高い値となった。しかし、予測できるとは言い難い結果となった。以下にデータ数 50000 における学習のグラフを示す。

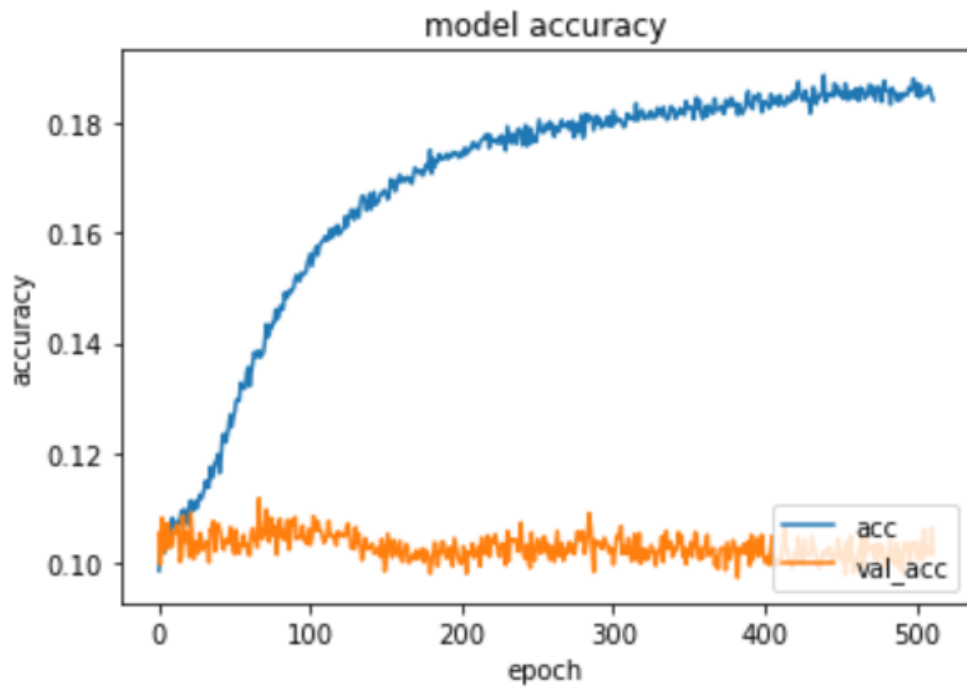
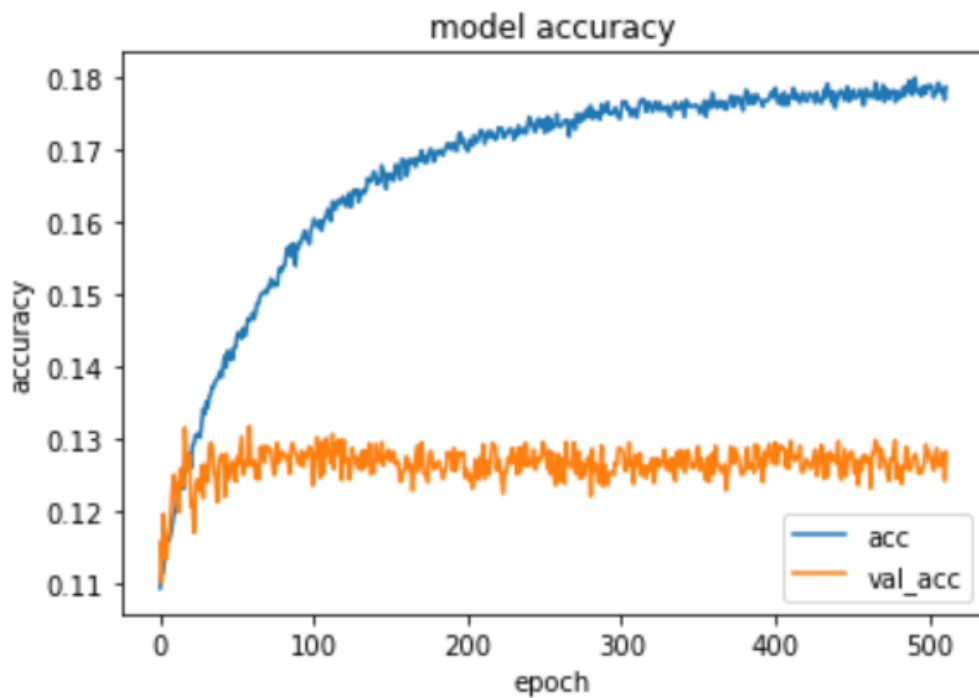
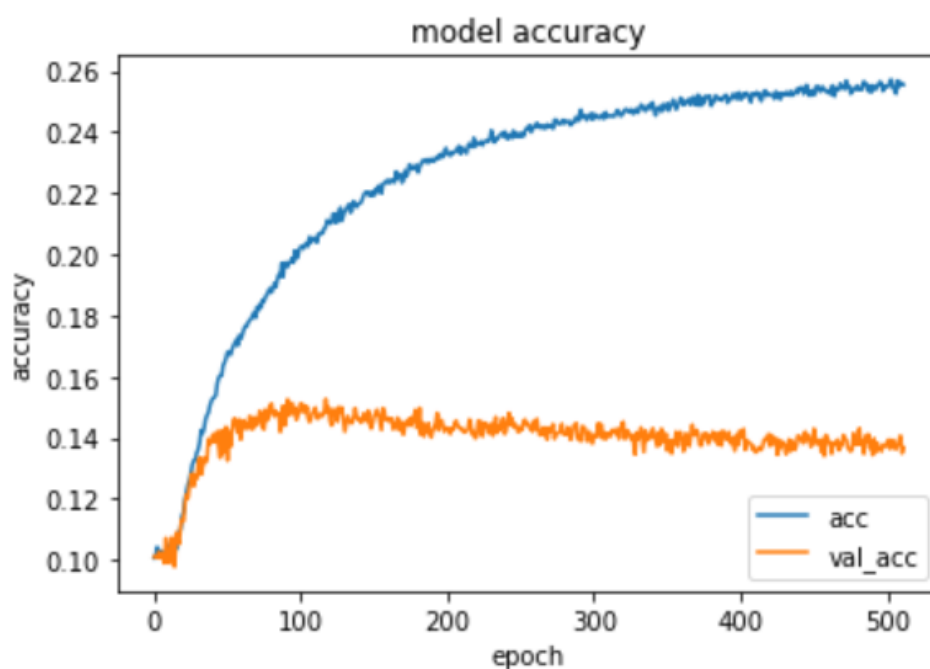
図6 $n=26$ 、データ数 50000 における予測精度図7 $n=26$ 、データ数 100000 における予測精度

表 5 関数 f の予測精度

データ数	予測精度
10000	0.1368
50000	0.1422
100000	0.1519

図 8 関数 f の予測精度

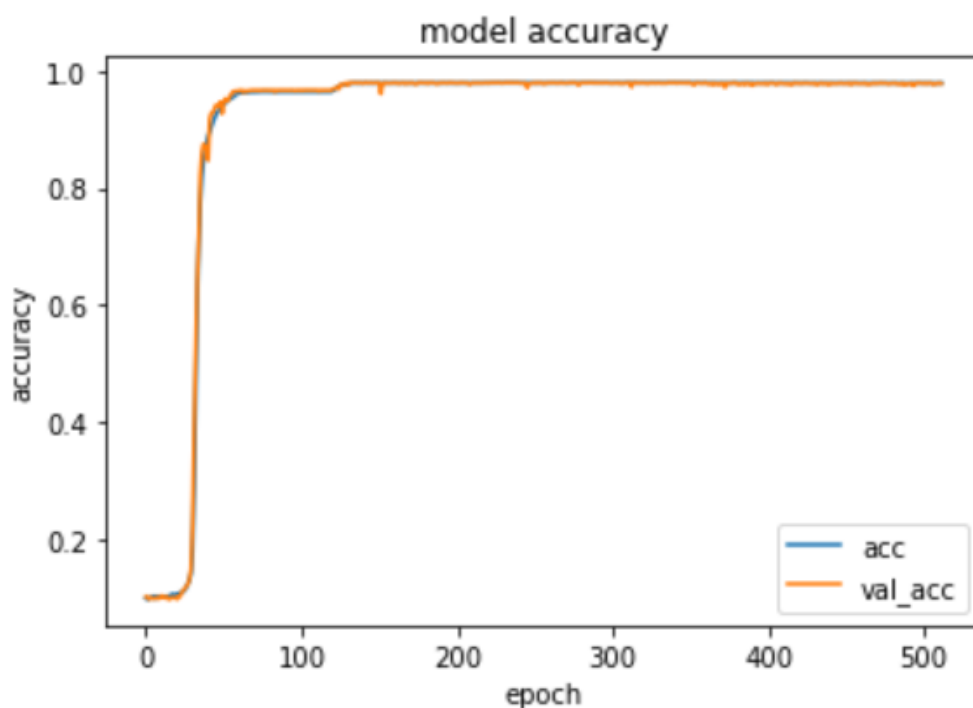
4.2.2 関数 g の予測精度

合成関数 $f \circ \sigma$ ならびに関数 f では、入力 $x_1, x_2 \cdots x_{12}, x_{13}$ において、 $x_{10} + x_{11} \bmod 10$ の計算結果によって、 $x_0 \sim x_9$ の値を決定し、そこに x_{12} と x_{13} を加えて、 $\bmod 10$ をとるということを行っていた。関数 g は、最初から $x_0 \sim x_9$ のどの値を加えるかが決まっている場合の関数である。表 6 は、一例として関数 g が $g = x_2 + x_{12} + x_{13}$ 、 $g = x_5 + x_{12} + x_{13}$ の場合の実験結果である。

これより、関数 g の予測精度は 95% を超える値となり、容易に予測できるという結果になった。また、このことから 14 個の入力から決まった 3 つの単純和を計算することは、LSTM での予測が可能であるということが分かった。以下に、 $g = x_2 + x_{12} + x_{13}$ 、データ数 50000 の時の学習グラフを示す。

表 6 関数 g の予測精度

関数 g	データ数	予測精度
$x_2+x_{12}+x_{13}$	10000	0.9634
$x_2+x_{12}+x_{13}$	50000	0.9779
$x_2+x_{12}+x_{13}$	100000	0.9864
$x_5+x_{12}+x_{13}$	10000	0.9678
$x_5+x_{12}+x_{13}$	50000	0.9781
$x_5+x_{12}+x_{13}$	100000	0.9842

図 9 関数 g の予測精度

4.3 関数 f の項の数を変化させた場合の予測精度

関数 f ならびに関数 g の予測精度の結果から、 $x_0 \sim x_{13}$ に 0 ～ 9 のランダムな数を代入しただけの場合では予測が難しく、決まった 3 つの単純和では予測が可能であるということが分かった。そこで、単純和ではない場合で関数 f よりも簡単な関数として、関数 h_1 について同様の実験を行った。

$$h_1 = x_j + x_{12} \bmod 10$$

$$j = x_{10} + x_{11} \bmod 10$$

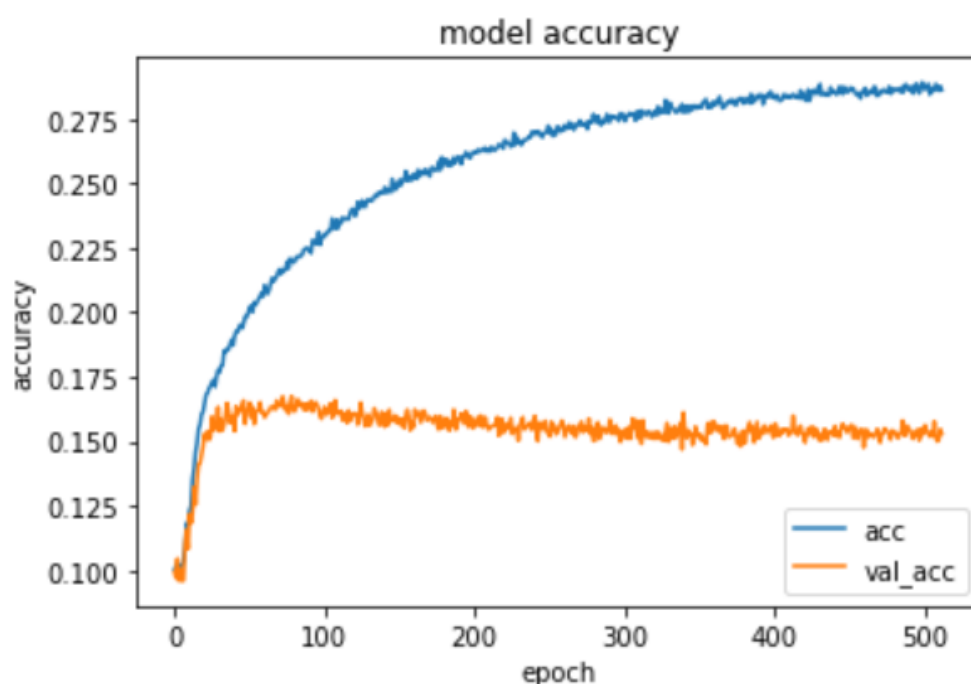
この時の結果を以下に示す。

$x_{10} + x_{11} \bmod 10$ の計算結果によって、 x_j に入る値が決まるという点は関数 f と同じだが、

表 7 関数 h_1 の予測精度

データ数	予測精度
10000	0.1396
50000	0.1529
100000	0.1611

関数 h_1 では、 x_j に加える項を x_{12} の 1 つだけに変更した。この結果、関数 f の予測精度より、数 % だけ上がったものの予測は難しいという結果になった。以下にデータ数 50000 の時の学習グラフを示す。

図 10 関数 h_1 の予測精度

また、関数 f の項の数を変更したものとして、関数 h_2 でも同様の実験を行った。

$$h_2 = x_j + x_{11} + x_{12} \bmod 10$$

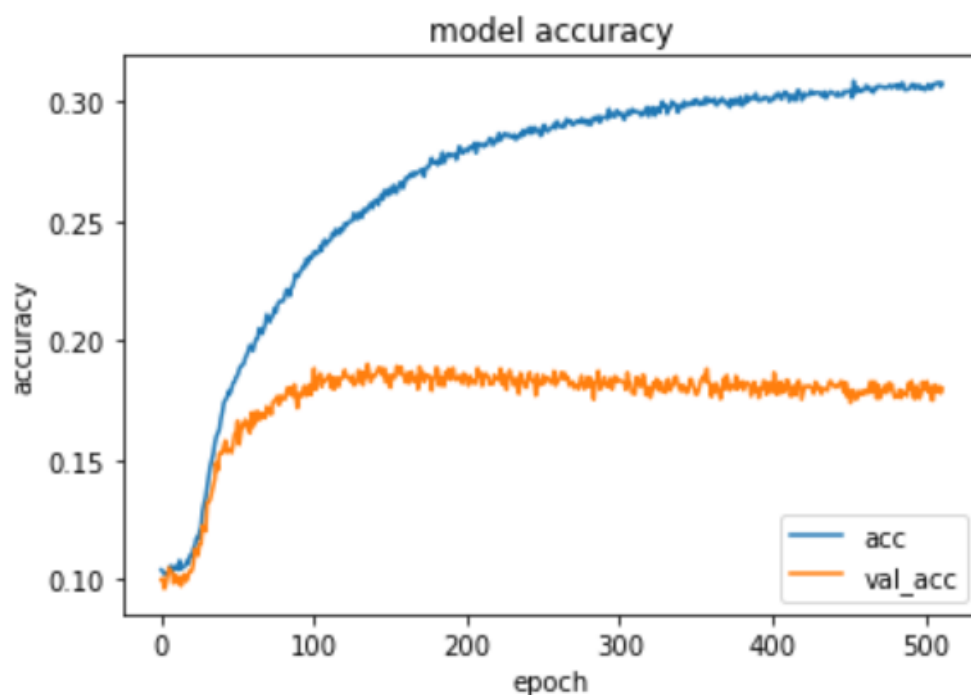
$$j = x_{10} \bmod 10$$

この時の結果を以下に示す。

関数 h_2 では、 x_j に入る値を決める際の計算が $j = x_{10} \bmod 10$ と項を 1 つだけに変更した。その後の x_j に加える項は x_{11} と x_{12} の 2 つであり、関数 f と同じである。この結果、関数 h_1 同様に予測精度は数 % の向上が見られた。しかし、予測は難しいという結果は変わらなかった。以下にデータ数 50000 の時の学習グラフを示す。

表 8 関数 h_2 の予測精度

データ数	予測精度
10000	0.1105
50000	0.1799
100000	0.1852

図 11 関数 h_2 の予測精度

5 結論

LSTM を用いて人間計算可能なパスワードの安全性の再評価を行った。結果としては、MLP を用いた既存研究と同様に予測はできないという評価となった。人間計算可能なパスワードにおいて提案された合成関数 $f \circ \sigma$ では、画像の枚数にかかわらず、予測精度は 10% 前後となり予測はできないという結果になった。また、マッピング関数 σ を除いて、関数 f だけにした場合も予測精度は 10% 代であったのに対して、決まった 3 つの単純和である関数 g では、90% を超える結果となった。さらに関数 f の項の数を変更した関数 h_1 、 h_2 についても、予測精度は 10% 代という結果となった。これらのことから、入力データ内で複数の演算が行われることや出力に関連する値が演算を行うたびに変わることが予測精度の低下につながっているということが明らかになった。今回、安全性評価を行った関数では、予測精度が 10% 代と 90% 代の極端な結果しか出なかった。今後の課題としては、予測精度が 40~50% となるような関数やマッピング関数の範囲を求め、深層学習の予測精度に影響を与

える要因を調査する。

謝辞

本研究を進めるにあたり、ご多忙にも関わらず、熱心にご指導いただきました櫻井幸一教授に心から感謝致します。また、研究にご協力いただきました櫻井研究室の皆様、特に村田壱生先輩に心から感謝いたします。

参考文献

- [1] AironWorks Survey&Research 働く人のサイバーセキュリティに関する実態調査
<https://www.aironworks.com/e-book/540/>
- [2] Manuel Blum, Santosh Vempala. The complexity of human computation via a concrete model with an application to passwords. Proceedings of the National Academy of Sciences, July 2017.
- [3] Jeremiah Blocki, Manuel Blum, Anupam Datta, Santosh Vempala. Towards Human Computable Passwords. Innovations in Theoretical Computer Science (ITCS). January 2017.
- [4] 村田壺生, 人間計算可能なパスワード認証に対する深層学習による安全性評価, 九州大学理学部物理学科情報理学コース卒業論文, 2022
- [5] PASONA デイープラーニング(深層学習)とは?
<https://www.pasonatech.co.jp/workstyle/column/detail.html?p=2181>
- [6] MathWorks デイープラーニング これだけは知っておきたい 3 つのこと
<https://jp.mathworks.com/discovery/deep-learning.html>
- [7] AUC RNN と LSTM の違いと特徴
<https://www.acceluniverse.com/blog/developers/2019/07/lstm.html>
- [8] aismiley 自然言語処理に使われる LSTM とは? https://aismiley.co.jp/ai_news/lstm/
- [9] AISIA LSTM Long Short Term Memory <https://products.sint.co.jp/aisia/blog/vol1-18>

付録

付録1：プログラミングコード

#モジュールの読み込み

```
from __future__ import print_function

import pandas as pd
from pandas import Series, DataFrame

from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

import numpy as np
import matplotlib.pyplot as plt

import keras
from keras.datasets import fashion_mnist
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten, LSTM
from tensorflow.keras.optimizers import RMSprop
from tensorflow.keras.optimizers import Adam

import tensorflow as tf
import random
import numpy as np
import pandas as pd
import csv
import os

n=26
Sgm = np.random.randint(0,10,n)
```



```
# define the secret mapping sigma
print(Sgm)

# Generate example pairs and save into the examples_pairs.csv file
data_size=50000
with open("train_pairs.csv", "w") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(["X0", "X1", "X2", "X3", "X4", "X5", "X6", "X7",
"X8", "X9", "X10", "X11", "X12", "X13", "Z"])
    #csvfile.close()
    for j in range(data_size):
        X = np.random.randint(0,10,14)#0～9 までの値を取るサイズ 14 の配列
        #Z=0
        mid=(X[10]+X[11])\%10
        Z=(X[mid]+X[12])\%10
        # Z is the value of f(sigma(X))
        example = np.append(X,Z)
        writer.writerow([example])
    csvfile.close()

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import seaborn as sns
import matplotlib.pyplot as plt
# import warnings
import warnings
# filter warnings
warnings.filterwarnings('ignore')

train = pd.read_csv("train_pairs.csv")
print(train.shape)
train.head()

#説明変数（ワインに含まれる成分）
```

```
#x = DataFrame(wine_data_set.drop("quality",axis=1))
x = train.drop(labels = ["Z"],axis = 1)

#目的変数 (各ワインの品質を 10 段階評価したもの)
#y = DataFrame(wine_data_set["quality"])
y = train["Z"]

# visualize number of digits classes
plt.figure(figsize=(15,7))
g = sns.countplot(y, palette="icefire")
plt.title("Number of digit classes")
y.value_counts()

#説明変数・目的変数をそれぞれ訓練データ・テストデータに分割
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2)

#データの整形

#y_train = keras.utils.to_categorical(y_train,10)
#y_test = keras.utils.to_categorical(y_test,10)

y_train =keras.utils.np_utils.to_categorical(y_train,10)
y_test =keras.utils.np_utils.to_categorical(y_test,10)

x_train = x_train.values
x_test = x_test.values

x_train = x_train.reshape(x_train.shape[0], x_train.shape[1], 1)
x_test = x_test.reshape(x_test.shape[0], x_test.shape[1], 1)
x_train.shape
print(x_train)
```

#ニューラルネットワークの実装

```
model = Sequential()
# model.add(Flatten())
# model.add(LSTM(32))
model.add(LSTM(32, input_shape = (14, 1)))

model.add(Dense(10, activation='softmax'))
print("\n")
```

#ニューラルネットワークの実装2

```
model.compile(loss='mean_squared_error',optimizer=RMSprop(),metrics=['accuracy'])
#勾配法には、Adam(lr=1e-3) という方法もある。
```

#ニューラルネットワークの学習

```
history = model.fit(x_train, y_train,batch_size=32,epochs=512,
verbose=1,validation_data=(x_test, y_test))
```

#ニューラルネットワークの推論

```
score = model.evaluate(x_test,y_test,verbose=1)
print("\n")
print("Test loss:",score[0])
print("Test accuracy:",score[1])
```

#10 段階評価したいワインの成分を設定

```
#sample = [7.9, 0.35, 0.46, 5, 0.078, 15, 37, 0.9973, 3.35, 0.86, 12.8]
#print("\n")
#print("--サンプルワインのデータ--")

#print(sample)
```

#ポイント：ワインの成分を Numpy の Array にしないとエラーが出る

```
#sample = np.array(sample)
#predict = model.predict_classes(sample.reshape(1,-1),batch_size=1,verbose=0)
```

```
#print("\n")
#print("--予測値--")
#print(predict)
#print("\n")

#学習履歴のグラフ化に関する参考資料
#http://aidiary.hatenablog.com/entry/20161109/1478696865

def plot_history(history):
    # print(history.history.keys())

    # 精度の履歴をプロット
    plt.plot(history.history['accuracy'])
    plt.plot(history.history['val_accuracy'])
    plt.title('model accuracy')
    plt.xlabel('epoch')
    plt.ylabel('accuracy')
    plt.legend(['acc', 'val_acc'], loc='lower right')
    plt.show()

    # 損失の履歴をプロット
    plt.plot(history.history['loss'])
    plt.plot(history.history['val_loss'])
    plt.title('model loss')
    plt.xlabel('epoch')
    plt.ylabel('loss')
    plt.legend(['loss', 'val_loss'], loc='lower right')
    plt.show()

# 学習履歴をプロット
plot_history(history)
```

付録 2：CAPTCHA との比較

人間計算可能なパスワードと同様に画像を使用する認証として、有名なものが CAPTCHA

である。CAPTCHA(Completely Automated Public Turing test to tell Computers and Humans Apart) は、メールアドレスの取得や会員登録フォーム、メッセージ投稿フォームなど広く用いられている認証方法である。CAPTCHA には大きく分けて 2 種類の方式が存在する。

1. 文字列 CAPTCHA：歪められたり、ノイズが加えられたランダムな文字や数字の列を表示し、ユーザーが文字列を読み取ることができるか否かによって人間とコンピュータを判別する。
2. 画像 CAPTCHA：複数の画像から、指定された対象が写っている画像のみを選択することによって識別する。

人間計算可能なパスワードと CAPTCHA では、画像を利用するという点においては共通しているが、人間とロボットの識別に利用されている CAPTCHA と、特定の個人とのパスワードとして利用される人間計算可能なパスワードは大きな違いがある。以下に比較を示す。

表 9 人間計算可能なパスワードと CAPTCHA の比較

	CAPTCHA	人間計算可能なパスワード
用途	認証	認証
画像の利用	○	○
認証の対象	人間またはロボット	特定の個人
事前学習	×	○
秘密	×	○
画像の利用目的	画像に写っているものの判別	計算に使用する数字の取得

人間計算可能なパスワードの大きな特徴の 1 つである事前学習を必要とするかどうか、人間計算可能なパスワードと CAPTCHA の最も大きな違いである。